

ESCALABILILIDAD Y COMPUTACIÓN DISTRIBUIDA CON SPARK

SPARK



Ivana Sánchez Pérez



ESCALABILIDAD Y COMPUTACIÓN DISTRIBUIDA CON SPARK

Introducción

La escalabilidad y la computación distribuida con Apache Spark constituyen pilares esenciales en el procesamiento masivo de datos. Spark es un sistema de computación distribuida diseñado para manejar grandes volúmenes de información de forma eficiente, rápida y tolerante a fallos.

Gracias a su arquitectura basada en memoria, Spark supera las limitaciones de sistemas como Hadoop MapReduce, alcanzando rendimientos hasta 100 veces superiores en determinadas operaciones. Esto lo convierte en una herramienta clave para aplicaciones de Big Data, aprendizaje automático, análisis en tiempo real y procesamiento de flujos de datos.

En esta tarea se presenta un análisis de escalabilidad horizontal mediante el uso de Apache Spark para distribuir el algoritmo de Strassen en la multiplicación de matrices.

Objetivos Específicos

- Configurar un cluster Spark local con múltiples workers
- Analizar la relación entre tamaño de problema y tiempo de ejecución
- Medir el speedup al aumentar el número de workers
- Comparar el rendimiento distribuido vs procesamiento local

Metodología y configuración

1- Entorno de pruebas

- **Sistema Operativo:** Windows 10/11
- **Java:** JDK 21
- **Python:** 3.11.9
- **Spark:** 4.0.1
- **Winutils.exe**
- **Cluster:** 1 Master + 3 Workers
- **Hardware:** 8 cores por worker, 6.8 GB RAM por worker

[illegible]

- Cluster local con 1 master

The screenshot shows the Spark Master web UI at `localhost:8080` and a terminal window. The UI displays the Spark Master at `spark://169.254.3.42:7077` with 0 workers. The terminal shows the command `C:\spark\spark-4.0.1-bin-hadoop3\bin>cd C:\spark\spark-4.0.1-bin-hadoop3\bin` and the output of the `spark-class` command, which starts the Spark Master daemon.

- Cluster local con1 master y 3 workers

The screenshot shows the Spark Master web UI at `localhost:8080` with 3 workers. The UI displays the Spark Master at `spark://169.254.3.42:7077` with 3 workers. The workers are listed in the 'Workers' section:

Worker ID	Address	State	Cores	Memory
worker-2025102112451-169.254.3.42-10388	169.254.3.42:10388	ALIVE	8 (0 Used)	6.8 GB (0.0 Used)
worker-2025102112451-169.254.3.42-11324	169.254.3.42:11324	ALIVE	8 (0 Used)	6.8 GB (0.0 Used)
worker-2025102112451-169.254.3.42-16037	169.254.3.42:16037	ALIVE	8 (0 Used)	6.8 GB (0.0 Used)

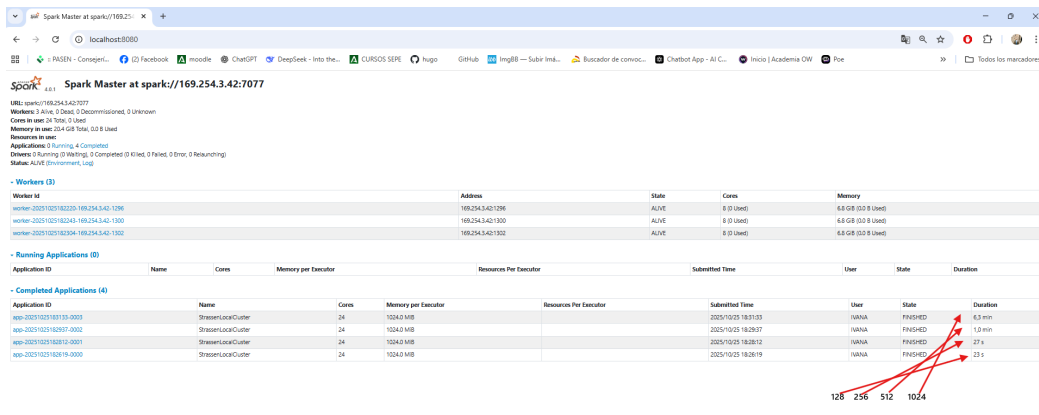
Ejercicios

Ejercicio 1: Cambiar el tamaño de la matriz N y comparar tiempos

Iniciamos el Master y abrimos en terminales independientes cada worker. Estas acciones las llevaremos a cabo desde ***C:\spark\spark-4.0.1-bin-hadoop3\bin***.

Desde otro terminal y ubicados donde se encuentre nuestro archivo `strassen_spark.py` procedemos a ejecutarlo con el comando:

Python strassen_spark.py cada vez que cambiemos N.



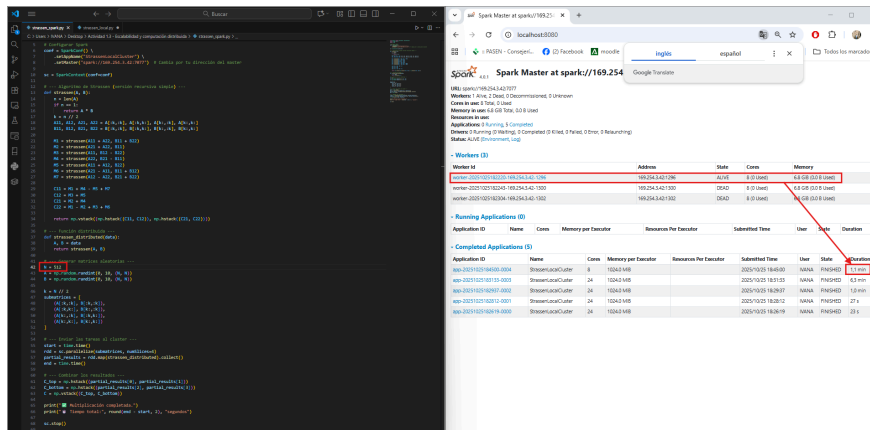
RESULTADOS DEL EJERCICIO 1

EJERCICIO 1	
Tamaño (N)	Tiempo (segundos)
128	21,92
256	25,81
512	60,57
1024	375,27

Análisis: Se observa un crecimiento exponencial del tiempo con respecto al tamaño, demostrando la complejidad $O(n^3)$ del algoritmo.

Ejercicio 2: Cambio del número de workers para ver el reparto de la carga

N=512 / 1 worker



N=1024 / 1 worker

The screenshot shows a Jupyter Notebook on the left with a Scala program that calculates the sum of squares of integers from 1 to 1024. The code is as follows:

```
def sumSquares(n: Int): Int = {  
  def loop(i: Int, acc: Int): Int = {  
    if (i > n) acc  
    else loop(i+1, acc+i*i)  
  }  
  loop(1, 0)  
}
```

On the right, the Spark Master web interface is shown. The 'Workers (3)' section lists three workers, with the first one highlighted: `worker-20231021162231-169.254.3.106`. The 'Running Applications (0)' and 'Completed Applications (6)' sections are also visible.

N= 512 / 3 workers

The screenshot shows a Jupyter Notebook on the left with a Scala program that calculates the sum of squares of integers from 1 to 512. The code is as follows:

```
def sumSquares(n: Int): Int = {  
  def loop(i: Int, acc: Int): Int = {  
    if (i > n) acc  
    else loop(i+1, acc+i*i)  
  }  
  loop(1, 0)  
}
```

On the right, the Spark Master web interface is shown. The 'Workers (3)' section lists three workers, with the first one highlighted: `worker-20231021162231-169.254.3.106`. The 'Running Applications (0)' and 'Completed Applications (6)' sections are also visible.

N= 1024 / 3 workers

The screenshot shows a Jupyter Notebook on the left with a Scala program that calculates the sum of squares of integers from 1 to 1024. The code is as follows:

```
def sumSquares(n: Int): Int = {  
  def loop(i: Int, acc: Int): Int = {  
    if (i > n) acc  
    else loop(i+1, acc+i*i)  
  }  
  loop(1, 0)  
}
```

On the right, the Spark Master web interface is shown. The 'Workers (3)' section lists three workers, with the first one highlighted: `worker-20231021162231-169.254.3.106`. The 'Running Applications (0)' and 'Completed Applications (7)' sections are also visible.

RESULTADOS DEL EJERCICIO 2

EJERCICIO 2 N=512/1024	
Workers	Tiempo (segundos)
1	62,81/413.17
3	65.67/359.52

Análisis: Spark sólo escala bien con problemas lo suficientemente grandes donde el tiempo del cómputo domina sobre el tiempo de comunicación.

Ejercicio 3: Sustituir el algoritmo por np.dot(A,B) y comparar el paralelismo Spark vs CPU local

Creamos un archivo nuevo de Python strassen_local.py y lo ejecutamos.

RESULTADOS DEL EJERCICIO 3

Ejercicio 3 N=512	
	234,73 segundos

Spark (3 workers): 65.67 segundos

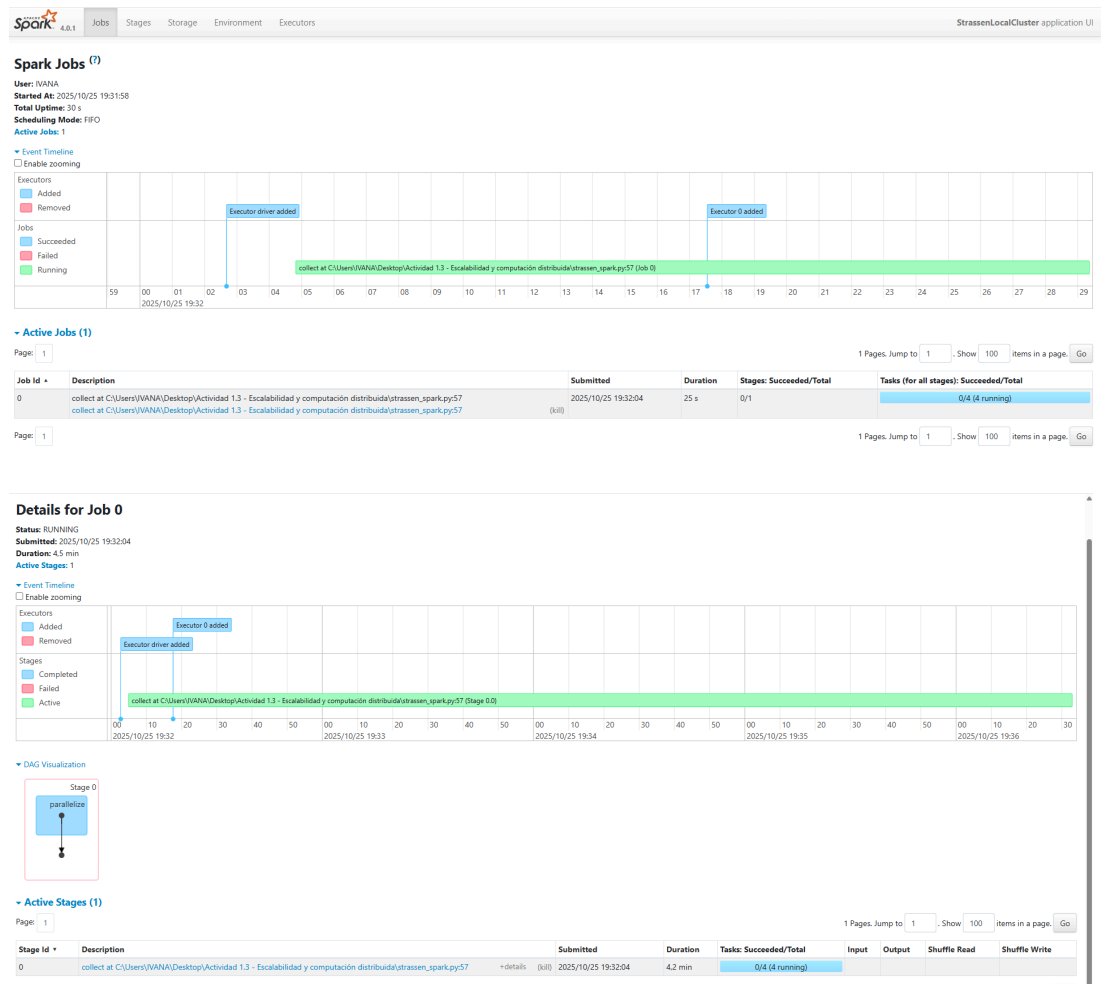
CPU Local: 234.73 segundos

Ganancia Spark: $234.73 / 65.67 = 3.57x$ (¡257% más rápido!)

Conclusión: Spark demostró ser **3.57 veces más rápido** que el procesamiento local secuencial.

Ejercicio 4: Observación del Spark Web UI, el DAG y las etapas

Spark solo muestra información detallada de aplicaciones en ejecución, por lo que para capturar dicha información hay que hacerlo durante la ejecución activa, ya que las aplicaciones completadas sólo muestran datos básicos.



ANÁLISIS DEL DAG Y ETAPAS:

1. DAG VISUALIZATION:

- Se observa una etapa simple "parallelize"
- Spark distribuye las submatrices entre los workers
- El algoritmo se ejecuta en paralelo en 4 tareas

2. STAGES:

- Stage 0: Procesamiento paralelo de Strassen
- 4 tasks distribuidas (0/4 running)
- Tiempo de ejecución: 4.2 minutos

3. DISTRIBUCIÓN:

- El trabajo se divide en 4 partes iguales
- Cada worker procesa una submatriz

- Combinación final de resultados en el driver

Ejercicio 5: Medir el speedup 1/3 workers

Fórmula: $\text{speedup} = \text{tiempo con 1 worker} / \text{tiempo con 3 workers}$

Para N=512

1 WORKER: 62.81 segundos

3 WORKERS: 65.67 segundos

Speedup = $62.81 / 65.67 = 0.96x$ (¡Más LENTO con más workers!)

Para N=1024

1 WORKER: 413.17 segundos

3 WORKERS: 359.52 segundos

Speedup = $413.17 / 359.52 = 1.15x$ (15% más rápido)

Resumen de la tarea

Se ha demostrado empíricamente que:

- Spark escala con problemas suficientemente grandes
- La distribución funciona pero tiene overhead
- El paralelismo acelera vs procesamiento secuencial
- La visualización es clave para entender ejecuciones distribuidas